Hairdresser Booking Platform: Handoff for the Next Model

As of 2026-10-17. Everything below is committed and pushed (HEAD `03b7703`).

Read this first. It is a multi-shop barbershop booking site in Persian (RTL). It runs on **Cloudflare Workers + D1**, and the code lives on GitHub. The owner is a single developer, Mohsen Iziy (GitHub `moh3niziy`). They write in Persian and expect Persian replies. They commit and push **themselves**: you must never run `git commit` or `git push`. Hand them a commit message and the command instead (see section 10).

---

1. What the product is

A booking and schedule tool that replaces a barbershop's paper appointment book.

- **Customers** open a shop's page and see the free time slots for each barber per day. They cannot book online; they **call** the barber to book.
- **Barbers / owners** use an admin panel to record appointments. They pick predefined services (name + duration) with one click, and the end time is computed automatically.
- **Site admin (the user, "Moh3n")** manages many shops: create, disable, delete, subscriptions, and leads.
- **Business model:** paid monthly subscriptions per shop. A public "join" page collects leads from shops that want their own site.

Pricing shown on the join page (Toman):

Plan	Price	Notes
Monthly	1,000,000	no long-term commitment
3 months	2,000,000	≈667k/month, 33% off
6 months	3,000,000	≈500k/month, 50% off, marked «به‌صرفه‌ترین»
Gift	+1 month free	for a shared Instagram post promoting the platform

The user said **"all plans are paid"** and that "1 month free" must **not** be advertised (it was removed from the join page). See Open item 1 about the automatic 30 free days still in the code.

---

2. Infrastructure

Thing	Value
Local path	`/Users/moh3niziy/Desktop/Flutter/Hairdresser` (the "Flutter" folder name is historical; there is no Flutter code)
GitHub	`https://github.com/moh3niziy/Hairdresser`, branch `main` (the user considered making it private; status unknown)
Hosting	Cloudflare Workers, worker name `hairdresser`, account `Moh3niziy@gmail.com`
Main URL	`https://hairdresser.moh3niziy.workers.dev`
Deploy	Cloudflare Workers Builds is connected to the GitHub repo: **every push to `main` auto-deploys** (deploy command `npx wrangler deploy`)
Database	Cloudflare D1, name `hairdresser-db`, id `9056ff39-043b-4252-8f00-5e2a8847fb00`, binding `DB`
Secrets (Cloudflare dashboard → Worker → Settings → Variables and Secrets, type **Secret**)	`ADMIN_USERNAME` = `Moh3n`, `ADMIN_PASSWORD` = known to the user (never write it into code or the repo)
Custom domain	`moghadamsbz.ir` + `www.moghadamsbz.ir` (zone on Cloudflare, nameservers `burt.ns.cloudflare.com` / `julissa.ns.cloudflare.com`, registered at IRNIC). It maps to shop `74260cf61a` («پیرایش مقدم»)
Support contact (shown to shops for renewals and sales)	phone `09399515814`, Telegram `@moh3nIziy` (constant `SUPPORT` in `worker.js`)

GitHub Pages was used early on and is now **unpublished**. Do not reintroduce it.

`wrangler.jsonc` (complete):

```
{
  "$schema": "node_modules/wrangler/config-schema.json",
  "name": "hairdresser",
  "main": "worker.js",
  "compatibility_date": "2025-09-01",
  "assets": { "directory": "./public", "binding": "ASSETS" },
  "d1_databases": [{ "binding": "DB", "database_name": "hairdresser-db", "database_id":
"9056ff39-043b-4252-8f00-5e2a8847fb00" }]
}
```

There is no `vars` block on purpose. Admin credentials are secrets, and a `vars` entry with the same name would conflict.

3. Repository layout

```
worker.js          (~590 lines) Cloudflare Worker: all /api routes, auth, D1 schema +
migrations
wrangler.jsonc     Worker config (assets + D1 binding)
public/index.html  Shell page: topbar, #app, #modalRoot, theme bootstrap, on-screen crash
reporter
public/app.js      (~1930 lines) whole front end (vanilla JS, no build step, no framework)
public/styles.css  (~640 lines) design tokens (light/dark), components, RTL, responsive
README.md          empty (the user cleared it on GitHub; leave it unless asked)
.gitignore
```

There are no npm dependencies and no build. Static files are served by Workers static assets. The Worker only runs for paths that do not match a file, which means `/api/*` and `/admin`.

`public/index.html` references assets with **absolute** paths (`/app.js`, `/styles.css`) so that `/admin/` also works.

---

4. Data model (D1)

The schema is created and migrated automatically on the first request (`migrate()` in `worker.js`). There are no migration files.

```
shops(id TEXT PK, name, location, data TEXT /*JSON*/, version INTEGER, created_at INTEGER,
      active INTEGER DEFAULT 1, telegram TEXT DEFAULT '', instagram TEXT DEFAULT '',
      expires_at TEXT DEFAULT '' /* 'YYYY-MM-DD', last active day, Tehran date */)
accounts(shop_id, username, pass /* 'salt$sha256' */, role /* owner|barber */, barber_id,
PK(shop_id, username))
meta(key PK, value)              -- 'secret' = random HMAC key for login tokens
leads(id PK, name, phone, note, source_shop, status /* new|contacted|done|rejected */, ip,
created_at)
```

- **`shops.data` JSON:** `{ barbers: [...], appointments: [...] }`.
- **barber:** `{ id, name, phone, owner?: true, hours: { "0".."6": {on, start:"HH:MM", end} }, services: [{id, name, minutes}] }`. The hours keys follow `Date.getDay()`: 0 = Sunday … 6 = Saturday. The UI shows the Iranian week starting Saturday.
- **appointment:** `{ id, barberId, date:"YYYY-MM-DD", start, end, customerName, customerPhone, services:[{id,name,minutes}], createdAt }`.
- **Optimistic concurrency:** the client sends `version`. The server does `UPDATE … WHERE version = ?`, and a mismatch returns 409. The client `mutate()` reloads, reapplies the change and retries up to 3 times.
- **Columns added later:** `active`, `telegram`, `instagram` and `expires_at` are added with `ALTER TABLE … ADD COLUMN` inside try/catch (a "duplicate column" error is ignored).
- **Legacy migration:** an old single-shop table `store` is migrated into shop id `main`. Shops with an empty `expires_at` get `today + 30 days`.
- **Pruning:** appointments older than 180 days are pruned on save (client side).

---

5. Roles, auth and permissions

Role	Logs in where	Can do
super (site admin)	Home page (`#/` or `/admin` on the main domain) with username + password checked against the `ADMIN_USERNAME` / `ADMIN_PASSWORD` secrets. A shop login form also accepts the super credentials	Everything: create, edit, disable, delete shops; set owner credentials; set subscriptions; read leads; enter any shop panel even if disabled or expired
owner	Shop page, username + password from `accounts`	Add, edit and delete barbers (including barber usernames/passwords); edit shop name, location, Instagram and Telegram, and their own credentials; manage all appointments; add themselves as a barber (`owner: true` barber, no separate account)
barber	Shop page, own account	Only their **own** appointments. The server ignores any other changes: it merges the barber's own appointments into the server copy and keeps the barbers list unchanged

- **Tokens:** HMAC-signed `base64url(payload).sig` with a 30-day expiry. The payload carries the role, shop and username plus a fragment of the password hash, so **changing a password invalidates old tokens**. Tokens are stored in `localStorage` key `hd_auth`.
- **Passwords:** salted SHA-256 (PBKDF2 was avoided because of the free plan's CPU limit). Usernames are `^[a-z0-9_.-]{3,32}$` and stored lowercase.
- **Persian digits:** input in Persian digits is normalised everywhere (`toEn`).

---

6. API (worker.js)

Public:

- `GET /api/shops` : the list. Disabled or expired shops are hidden unless the requester is super; super also gets `sub`.
- `GET /api/shops/:id` : the customer view. Customer names and phones are stripped and only busy ranges are sent; the subscription status is stripped. An expired shop returns 403 `{expired, support, shopName}` **even for super**. A disabled shop returns 403 except for super.
- `POST /api/shops/:id/login` : owner/barber login (super credentials also accepted). Returns 403 if the shop is disabled or expired.
- `POST /api/login` : super login. Returns 503 if the secrets are missing.
- `GET /api/support` : returns `{support}` .
- `POST /api/leads` : the join-page form. Has a honeypot field `website` , a limit of 5 per IP per hour, and Iranian phone validation.

Super only:

- `POST /api/admin/shops` : create a shop with owner credentials. Sets `expires_at = today + 30` .
- `DELETE /api/admin/shops/:id` .
- `PUT /api/admin/shops/:id/status {active}` .
- `PUT /api/admin/shops/:id/subscription {expiresAt}` .
- `GET|PUT|DELETE /api/admin/leads[/:id]` .

Shop scoped (owner or super):

- `PUT /api/admin/shops/:id` : name, location, telegram, instagram, owner credentials.
- `PUT|DELETE /api/admin/shops/:id/accounts/:barberId` : barber login.
- `GET|PUT /api/admin/shops/:id/data` : the barber role can also use this, with the merge rules above.

Not an API route:

- `GET /admin` and `/admin/` : the Worker serves `index.html` .

Server-side rules worth knowing:

- **No bookings in the past.** A new or moved appointment must not be on a past date or start earlier than now – 5 minutes. The reference is **Asia/Tehran** time. Renaming or deleting past appointments is allowed.
- **Disabled or expired shops** accept **no new appointments**, not even from super. Edits and deletes are allowed.
- **Grace period.** During the 7 days after the subscription ends, owner and barber writes are rejected (403 `readonly`); super writes are allowed.

---

7. Subscription logic

`subStatus(expires_at)` , using the Tehran date:

State	Condition	Effect
`active`	more than 7 days left	normal
`warning`	0–6 days left (day 0 = last day)	yellow banner in the owner panel with support contacts
`grace`	1–7 days **after** expiry	owner and barbers can log in but **read-only** (UI hides edit buttons, server rejects writes). Red banner with countdown. The customer page still works
`expired`	more than 7 days after expiry	exactly like a manual disable: hidden from the list, logins blocked, customer page shows a centred message («اشتراک این آرایشگاه به پایان رسیده است») with Telegram, call and SMS buttons

- **Subscription dialog (home, super):** the calendar button on a shop card. Options are +1/3/6/12 months, a custom number of days, or an exact **Jalali** date. Extensions start from the current end date, or from today if already expired. Jalali conversion is `toJalali` / `fromJalali` in `app.js` , using Intl with a day-by-day correction; it was tested over 2,000 days.
- **Badges on cards:** «مهلت تمدید X روز مانده», «X روز · منقضی از …» or «….

---

8. Front-end behaviour (public/app.js)

Routing (hash based, plus pathname for `/admin`):

- `#/` : the shop list (home).
- `#/s/<shopId>` : the customer page.
- `#/s/<shopId>/admin` : the shop panel.
- `#/join[/<sourceShopId>]` : the lead and pricing page.
- `/admin` on the main domain opens the super login. On a custom domain it opens that shop's panel.

Custom domains: the `DOMAIN_SHOPS` map near the router (`'moghadamsbz.ir': '74260cf61a'`). On such a domain `#/` shows that shop, and the shop list is not reachable. Add new domains there and connect them in

Cloudflare → Worker → Domains & Routes → Custom domain.

Hidden admin entry: there is **no admin button** anywhere. You **long-press the scissors logo for 2 seconds** (a ring fills up). On home this opens the super login; on a shop page it opens the shop login or panel. The user explicitly asked that shop pages have **no way back** to the shop list (no home icon).

Customer page:

- Top bar: icons for location (Google Maps search or the saved map URL), Telegram and Instagram, each only if set, plus the theme toggle.
- Date strip: next 21 days, in Persian.
- Timeline widget: a status sentence such as «الان خالی است تا ساعت …», a «الان» pill, an hour ruler with start/end labels, free/booked/past segments, and past time hatched.
- Free-slot list, a call card, and services with durations.
- The barber picker is **hidden when the shop has only one barber**.
- The bottom promo link «تو هم می‌خوای یک سایت مثل «نام آرایشگاه» داشته باشی؟ کلیک کن» opens `#/join/<shopId>`.
- The tab title is just the shop name.

Admin panel:

- Tabs «نوبت‌ها» and «اطلاعات آرایشگاه» (barbers, the owner-as-barber section, shop info with social links and owner credentials, backup and restore).
- Schedule: agenda of appointments plus free-gap buttons and a floating «نوبت جدید» button. Past days and past hours are locked, and the admin timeline hatches past time but keeps past bookings visible as faded red.
- Appointment dialog: one-click service chips, start-time picker, suggested starts, live end time, conflict and past-time checks.

Home (super): tabs «آرایشگاه‌ها» and «درخواست‌ها» (leads, with a red count of new ones). Shop cards have subscription, edit, enable/disable and delete buttons.

Join page:

- Hero, contact buttons at the top and bottom, the pricing cards, the Instagram gift box, 4 feature cards and the lead form.
- Auto-refresh is disabled here so typing is not lost.

General:

- Auto-refresh every **3 minutes** when no modal is open and the tab is visible.
- Light/dark theme stored in `localStorage` `hd_theme`. The Vazirmatn font comes from jsDelivr. Everything is RTL and in Persian, with Persian digits.
- **Mobile-compatibility rules** learned the hard way:
- No numeric separators (`1_000_000`), no `.at()`, no `structuredClone`.
- Never position elements far off-screen (`left:-9999px` made the RTL page 10,000 px wide, and phones zoomed out to a blank-looking page).
- `html { overflow-x: clip }` is a safety net.
- `index.html` has an on-screen crash reporter that replaces a blank page with the error and user agent.

---

9. How things were tested (reproduce this)

There is no test suite. Testing was done in **headless Chrome** (`/Applications/Google Chrome.app`) with a harness:

- Copy `public/*` and `worker.js` into a scratch folder.
- Inject into `index.html` a script that loads **sql.js 1.10.3 from cdnjs** as a fake D1 (`prepare/bind/first/all/run/batch`).
- Override `window.fetch` so that `/api/*` calls `worker.fetch(request, {DB, ADMIN_USERNAME:'Moh3n', ADMIN_PASSWORD:'…', ASSETS:{fetch}})`.
- Serve with `python3 -m http.server` and drive it from a wrapper page through `iframe.contentWindow`.
- Use `--dump-dom` for logs and `--screenshot` for visuals. Headless Chrome's minimum window width is 500, so wrap pages in a 375 px iframe for phone width.

Node v26 is now installed: `node --check public/app.js` and `node -e "import('./worker.js')"` are quick syntax checks. Wrangler is not installed locally.

The user cannot see your scratchpad. It gets wiped between sessions, so rebuild the harness when needed.

---

10. Working agreements with the user (important)

1. **Reply in Persian.** UI text is Persian. Keep code comments in Persian, matching the existing style.
2. **Never commit or push.** After each change, give a short English commit message and the one-line command, for example:

   ```
   git add -A && git commit -m "Short imperative message" && git push
   ```

   Saved memory notes: *user-commits-themselves*, *no-commit-attribution*.
3. **No AI attribution anywhere.** No `Co-Authored-By`, no "Generated with" lines. The first commit was rewritten to remove one.
4. **For large requests, split the work into tasks** and do them one by one. The user said this explicitly for the subscription feature.
5. **The user asks questions before deciding.** When they say they will ask first, answer only; do not implement.
6. **Explain Cloudflare dashboard steps concretely.** The user follows them step by step and sends screenshots; earlier they put a secret's value in the name field.
7. **Do not put secrets in the repo.** The repo may be public; the old password `1234` is still in git history but no longer works.

---

11. History (what was tried and why it changed)

1. A Python server plus a static site was rejected: the user wanted no server.
2. A pure static site on GitHub Pages with `data.json` came next. Writes via the GitHub API with a token were built and then **removed** at the user's request ("don't connect to GitHub"). It then used `localStorage` plus manual `data.json` export, which could not sync between devices.
3. **Move to Cloudflare Workers + D1** (current). Builds come from GitHub.
4. Multi-shop list and per-shop pages, then owner/barber accounts with role-based access, then site-admin credentials as secrets and shop disabling.
5. UX work: long-press admin entry, no back link from shop pages, a redesigned timeline, social links in the top bar, the barber picker hidden for single-barber shops, and no bookings in the past.
6. The custom domain `moghadamsbz.ir` and the `/admin` path.

7. Subscriptions (30 free days, warning, 7-day read-only grace, auto-expiry), then the admin subscription UI with Jalali dates, then the centred expired page.
8. The join/lead page with pricing, a promo link on every shop page, and a leads inbox in the super admin. Then mobile fixes (the blank page was caused by the off-screen honeypot).

The messy commits `09ba356` and `0822f18` have a whole shell command as their message. The user chose not to rewrite them.

---

12. Open items and caveats

1. **The automatic 30 free days for new shops conflict with "all plans are paid".** `FREE_DAYS = 30` in `worker.js` is applied at creation and in the legacy migration. Asked, not yet decided. The super can always set the date manually.
2. **Grace period is 7 days**, but the user once referred to "5 days". Confirm before changing `GRACE_DAYS`.
3. Two Cloudflare builds failed earlier (`ccbf101`, `d7abf7b`) for an unknown reason. Later builds succeeded. If a deploy does not appear, ask the user for the build log («Copy build log»).
4. Whether the GitHub repo is private is unknown. If it is made private, the Cloudflare GitHub app needs access to the repo.
5. There is no online payment. Renewals go through contact with the user (Telegram, phone or SMS), and then the super sets the date in the subscription dialog.
6. Shop owner credentials are set by the super when creating or editing a shop. Shops created before accounts existed have no owner until the super sets one.
7. There is no automated test suite or CI. Use the harness in section 9.